

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»
Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

ОТЧЁТ
по практической работе № 2
«Основы Python: коллекции, функции, циклы
и расширенные возможности Python»

Дисциплина: Технологии разработки приложений на базе фреймворков

Семестр: 5 семестр, 2026–2027 учебный год

Студент: Думлер Виктория

Группа: _____

Преподаватель: Ящун Татьяна Викторовна

Год выполнения: 2026

1. Выполненные материалы Яндекс Практикума

Материалы курса «Бэкенд-разработка» выполнены в полном объёме, предусмотренном методическими указаниями к ПР2. Темы раздела 01, перенесённые с ПР1, завершены; выполнен проект «Холодильник»; изучены уроки раздела 03 (кроме декораторов — они относятся к ПР3).

Раздел	Выполненные темы
Раздел 01. «Основы Python» (продолжение)	Коллекции; Итерации и циклы; проект «Холодильник»
Раздел 03. Тема «Python: новый уровень»	«Новая задача»; «Пользовательские модули и пакеты»; «Интроспекция»; «Документирование кода»; «Исключения»; «Доработка проекта»; «Работа с файлами»; «Контекстные менеджеры»; «Новый интерфейс для проекта»; «Практика по теме»
Раздел 03. Тема «Расширенные возможности Python»	«Пространство имён и области видимости»; «Итераторы и генераторы»; «Lambda-функции»

2. Индивидуальный проект

Название: Система управления пользовательскими ролями (RBAC — Role-Based Access Control).

Начальный сценарий ПР1 не удалён, а переработан: одиночные переменные пользователя, роли и группы стали коллекциями, функции `check_permission`, `describe_role`, `format_user_card` и `convert_age` сохранены и используются вместе с новыми операциями.

Функции, реализованные на ПР2:

- просмотр, поиск и сортировка пользователей;
- добавление пользователя и назначение роли;
- проверка разрешения (функция ПР1 + проверка по JSON);
- просмотр ролей, разрешений и групп;
- добавление пользователя в группу и исключение из группы;
- статистика по ролям и используемым разрешениям;
- загрузка и сохранение данных в JSON;
- обработка некорректного ввода и ошибок файлов.

Структура модулей:

Файл	Назначение
main.py	Точка запуска, меню, вывод данных
users.py	Пользователи, карточка и возраст из ПР1
roles.py	Роли, разрешения, check_permission из ПР1
groups.py	Группы и членство пользователей
storage.py	Чтение и запись JSON через with
utils.py	Ввод, идентификаторы, интроспекция
data/*.json	Пользователи, роли, разрешения, группы
tests/	Автотесты pytest (13 тестов)

Формат хранения данных: JSON-файлы в каталоге data/. В памяти коллекции хранятся как словари dict[int, dict], на диске — как списки словарей.

Репозиторий: <https://github.com/VikaD2345/-User-role-management-system>

3. Результат работы

В результате выполнения ПР2 завершены темы раздела 01 и выполнен раздел 03 Яндекс Практикума (кроме декораторов). Код индивидуального проекта разделён на модули; данные переведены в JSON-файлы; реализованы поиск, сортировка, статистика, проверка прав, работа с группами и обработка исключений. Код проверен flake8, основные функции снабжены аннотациями типов, pytest: 13 passed.

4. Вывод

Практическая работа позволила перейти от одноразового сценария ПР1 к функциональному консольному приложению. Коллекции, функции, модули, исключения и файлы применены к управлению ролями: программа работает с несколькими пользователями, сохраняет изменения и проверяет доступ. Это готовит проект к объектной модели на ПР3. Классы на этом этапе не использовались — так требует методичка.

КОНТРОЛЬНЫЕ ВОПРОСЫ

Практическая работа № 2

«Основы Python: коллекции, функции, циклы и расширенные возможности Python»

Думлер Виктория · тема — система управления пользовательскими ролями

Вопросы сгруппированы по пяти блокам методички ПР2. Ответы краткие и привязаны к коду индивидуального проекта, чтобы по ним можно было отвечать на защите.

По коллекциям, циклам и функциям

1. Какие виды коллекций Python вы знаете? Для каких задач предназначен каждый из них?

Ответ. Четыре основные: `list` — упорядоченный изменяемый набор (пользователи, результаты поиска); `tuple` — фиксированный набор (пара «роль, группа» в `resolve_names`); `set` — уникальные элементы (коды разрешений роли); `dict` — пары «ключ — значение» (пользователи, роли и группы по `id`).

2. Чем список отличается от кортежа?

Ответ. Список можно менять: добавлять пользователя, менять `role_id`. Кортеж неизменяемый, его удобно вернуть как пару значений. В проекте список — найденные пользователи, кортеж — (`role_name`, `group_name`).

3. В каких случаях удобно использовать множество?

Ответ. Когда нужны уникальные значения и быстрая проверка «есть / нет». Функция `get_permission_codes` собирает `set` кодов разрешений роли, а статистика объединяет множества через `|=`, чтобы показать, какие права реально используются.

4. Почему словари называются коллекцией пар «ключ — значение»?

Ответ. Каждый элемент — ключ и связанные с ним данные. В проекте ключ — `id`, значение — словарь с полями записи. Так пользователя можно взять за $O(1)$: `users[user_id]`, а не перебирать весь список.

5. Чем цикл `for` отличается от цикла `while`?

Ответ. `for` перебирает коллекцию: пользователей, роли, пункты справки. `while` крутится, пока условие истинно: главное меню и повтор ввода в `input_int` / `input_date`, пока пользователь не введёт корректное значение или не выберет выход.

6. Для чего используется функция `range()`?

Ответ. Чтобы получить последовательность целых чисел, когда нужен индекс, а не сам элемент. В ПР2 основной перебор идёт по `.values()` словарей. `range()` использовался в упражнениях Практикума и пригодится, если понадобится нумерация с нуля.

7. Как организовать перебор элементов словаря в цикле?

Ответ. `for key in d` — ключи; `for value in d.values()` — значения; `for key, value in d.items()` — пары. В `show_users` перебираются `users.values()`, в проверке разрешений — `permission_ids` роли.

8. Для чего используются операторы `break` и `continue`?

Ответ. `break` сразу выходит из цикла: в `main()` при выборе «0» цикл меню завершается. `continue` переходит к следующей итерации, пропуская остаток тела. Нужны, чтобы не городить лишние вложенные `if`.

9. Как объявить функцию в Python? Что возвращает функция без оператора `return`?

Ответ. `def имя(параметры):` и тело с отступом. Без `return` функция возвращает `None`. Так устроены `show_users` и `persist`: они печатают или сохраняют данные, а не отдают значение вызывающему коду.

10. Чем позиционные параметры отличаются от именованных?

Ответ. Позиционные идут по порядку: `add_user(users, name, age, ...)`. Именованные задаются явно: `sort_users(users, field="age")`. Именованные делают вызов понятнее и позволяют пропускать аргументы со значением по умолчанию.

11. Что такое область видимости переменной?

Ответ. Это место, где имя видно программе. Локальные переменные функции (`user`, `found`) снаружи недоступны. Глобальные — пути к `JSON` в `main.py`. Поэтому данные передаём аргументами, а не читаем чужие локальные переменные.

12. Что такое `docstring` и для чего он нужен?

Ответ. Строка документации сразу под заголовком функции. В проекте у основных функций есть `docstring`: его читает человек и функция `describe_function` через `inspect.getdoc` — пункт меню «Справка по функциям».

По модулям, исключениям и файлам

13. Что такое модуль Python?

Ответ. Файл `.py` с функциями и данными, который можно импортировать. В проекте модули разделены по смыслу: `users`, `roles`, `groups`, `storage`, `utils`, точка входа `main`.

14. Чем импорт модуля отличается от импорта отдельного объекта из модуля?

Ответ. `import users` подключает весь модуль: `users.add_user(...)`. `from users import add_user` берёт только функцию. Во втором случае вызов короче, но нужно следить, чтобы имена не пересекались.

15. Что такое пакет и для чего нужен файл `__init__.py`?

Ответ. Пакет — каталог с модулями. `__init__.py` помечает каталог как пакет и может реэкспортировать объекты. В ПР2 пакет не собирали: модули лежат в корне проекта, этого достаточно.

16. Для чего используется конструкция `try/except`?

Ответ. Чтобы перехватить ошибку и продолжить работу. В `convert_age` ловится `ValueError`, в `load_json` — `FileNotFoundError` и `JSONDecodeError`, во вводе — некорректное число или дата. Без `try` программа упала бы на первой опечатке.

17. Чем блок `finally` отличается от блока `else`?

Ответ. `else` после `try` выполняется, только если исключения не было. `finally` выполняется всегда — и при успехе, и при ошибке. Контекстный менеджер `with` как раз гарантирует закрытие файла по принципу `finally`.

18. Для чего используется оператор `raise`?

Ответ. Чтобы самому возбудить исключение, если данные нарушают правило. В ПР2 ошибки ввода обрабатываем без `raise` наружу: повторяем запрос. `raise` понадобится позже, когда появятся собственные типы ошибок предметной области.

19. Зачем закрывать файл после работы с ним?

Ответ. Иначе данные могут не записаться на диск, а дескриптор останется занятым. При записи JSON это особенно важно: несохранённый пользователь «пропадёт» после перезапуска.

20. Какие преимущества даёт контекстный менеджер `with`?

Ответ. `with open(...) as file`: сам закроет файл, даже если `json.load` выбросил ошибку. В `storage.py` так устроены `load_json` и `save_json` — это требование методички к ПР2.

По расширенным возможностям языка

21. Что такое интроспекция? Какие функции Python для нее используются?

Ответ. Возможность смотреть на объект во время выполнения: имя, сигнатура, документация. В `utils.describe_function` используются `inspect.signature` и `inspect.getdoc`. Пункт меню 12 показывает справку по ключевым функциям проекта.

22. Что такое пространство имен?

Ответ. Словарь имён, доступных в данном месте программы. У модуля своё пространство (функции `users.py`), у функции — локальное. Поэтому `add_user` в `users.py` и `handle_add_user` в `main.py` не мешают друг другу.

23. Что такое итератор и генератор?

Ответ. Итератор отдаёт элементы по одному через `next()`. Генератор — функция или выражение, которое создаёт итератор. `filter_users_by_role` возвращает генераторное выражение: пользователи с нужной ролью не копируются в новый список сразу.

24. Для чего используется оператор `yield`?

Ответ. Он превращает функцию в генератор: значение отдаётся, а состояние функции сохраняется до следующего вызова. В проекте хватило генераторного выражения; `yield` — тот же приём, только в виде функции.

25. Что такое лямбда-функция и где она применяется?

Ответ. Короткое безымянное выражение. В `sort_users` лямбда задаёт ключ сортировки: `key=lambda item: item.get(field, "")`. Так можно сортировать и по имени, и по возрасту без отдельной именованной функции.

По качеству кода и тестированию

26. Что такое аннотации типов и как они оформляются в сигнатуре функции?

Ответ. Подсказки типов параметров и результата: `def add_user(users: dict[int, dict], name: str, age: int) -> dict`. Они не меняют выполнение программы, но делают договор о данных явным. Основные функции проекта аннотированы.

27. Для чего используется линтер flake8 и как выполнить проверку кода проекта?

Ответ. flake8 проверяет PEP 8: длина строки, лишние импорты, пробелы. Запуск из корня проекта: `flake8`. Если замечаний нет, линтер ничего не печатает. Перед сдачей ПР2 проверка проходит без замечаний.

28. Какие требования к оформлению кода предъявляются в рамках ПР2?

Ответ. Модули по смыслу, понятные имена, PEP 8, нет мёртвого кода и дублирования, docstring и аннотации у основных функций, проверка flake8. Повторяющаяся логика вынесена в `next_id`, `load_json` и `persist`.

29. Как устроен автоматизированный тест pytest и что проверяет оператор assert?

Ответ. Тест — функция `test_...:` готовит данные, вызывает функцию проекта, проверяет результат через `assert`. Если условие ложно, тест падает. Один тест — одна функция — один сценарий, как в методичке.

30. Как запустить тесты проекта и как интерпретировать результат их выполнения?

Ответ. Из корня проекта команда `pytest`. Все тесты должны быть `passed`. Сейчас 13 тестов: пользователи, роли, группы и запись JSON. `failed` означает, что после правки кода нужно смотреть имя теста и сообщение `assert`.

По индивидуальному проекту

31. Какие функции реализованы в проекте на этапе ПР2?

Ответ. Просмотр, поиск и сортировка пользователей; добавление пользователя; назначение роли; проверка разрешения; просмотр ролей и групп; включение в группу и исключение; статистика; справка через интроспекцию. Из ПР1 сохранены `check_permission`, `describe_role`, `format_user_card` и `convert_age`.

32. Как в проекте представлены данные (структуры, формат хранения)?

Ответ. В памяти — словари `dict[int, dict]` по `id`. На диске — JSON-списки в `data/`: `users.json`, `roles.json`, `permissions.json`, `groups.json`. У пользователя есть `role_id` и `group_id`, у роли — список `permission_ids`.

33. Как устроена проверка доступности помещения?

Ответ. В сквозном примере методички это `is_room_available`. В моём проекте аналог — проверка разрешения. `check_permission(role, permission)` осталась из ПР1: администратор может всё, менеджер — всё кроме удаления пользователей, сотрудник — только просмотр. `role_allows_permission` дополнительно смотрит `permission_ids` роли в JSON.

34. Какие исключения обрабатываются в программе и зачем?

Ответ. `ValueError` — нечисловой ввод и неверная дата; `FileNotFoundError` — нет JSON при первом запуске; `JSONDecodeError` — повреждённый файл. Цель одна: не дать программе завершиться аварийно и объяснить, что исправить.

35. Какие модули входят в состав проекта и как распределена логика между ними?

Ответ. `main.py` — меню; `users.py` — пользователи и карточка; `roles.py` — роли и проверка прав; `groups.py` — группы; `storage.py` — JSON; `utils.py` — ввод, `next_id`, интроспекция. Тесты лежат в `tests/`.

36. Какие конструкции Python применены в реализованных функциях?

Ответ. Словари, списки, множества, кортеж; циклы `for` и `while`; функции с аннотациями и `docstring`; генератор `filter_users_by_role`; `lambda` в `sort_users`; `try/except`; `with + json`; `inspect` для справки.

37. Какие части проекта будут изменены после изучения ООП на ПР3?

Ответ. Пользователь, роль, разрешение и группа станут классами. Проверки уйдут в методы объектов, а не в отдельные функции над словарями. JSON пока останется, позже его сменит база данных.

38. Почему данные проекта хранятся в файлах, а не в переменных программы?

Ответ. Переменные живут только до закрытия программы. Файлы сохраняют пользователей, роли и группы между запусками. Это уже поведение настоящего приложения, а не одноразовый сценарий ПР1.

Репозиторий проекта: <https://github.com/VikaD2345/-User-role-management-system>